Technical Document

Manage Workflow — SRS

SOFTWARE REQUIREMENTS SPECIFICATION

Manage Workflow · Workflow Designer

Software Requirements Specification (SRS) — Manage Workflow

Feature: Manage Workflow (Workflow designer) — Apex controller: force-app/main/default/classes/controller/manageWorkflow/ManageWorkflowPageController.cls — LWC: force-app/main/default/lwc/manageWorkflow/ — Date: 2026-06-22

1. Purpose & Scope

The Manage Workflow page is the workflow-design surface of the Jira Clone. For a selected Project it lets a user author and maintain the workflows that govern how tickets move between statuses. From this page a user can:

- See the list of workflows defined for the selected project and open one to edit, or create a new one.
- See the chosen workflow as a visual graph: one node per project status and one curved arrow per transition.
- Create new statuses (nodes) for the project.
- Create a transition between two statuses by clicking the source then the target node.
- Inspect a transition (name, from/to status, record status, created date) and activate or delete it.
- Attach field-validation rules to a transition (the rules a ticket must satisfy to take that transition).
- Publish the draft by updating the workflow, which activates every pending transition at once.

The project is selected once and persisted in localStorage('projectId'); if absent the page shows the Choose Project splash. The chosen workflow is persisted in localStorage('workflowId'); the three entry states are: (1) no project → Choose Project, (2) project but no workflow → workflow list, (3) workflow selected → the visual editor.

2. Actors

Actor	Description
Project Member (User)	The authenticated Salesforce user designing the project's workflows — creates statuses and transitions, attaches validation rules, and activates or deletes transitions.

3. Functional Requirements

3.1 Project Selection and Workflow List

- FR-1 The system shall allow a user to select a project before accessing any workflow editor.

- FR-2 The system shall load and list the workflows defined for the selected project (name and record status).
- FR-3 When the project has no workflows, the system shall show an empty-state message inviting the user to create one.
- FR-4 The system shall allow a user to create a new workflow by name; on success the new workflow opens directly in the editor.

3.2 Workflow Editor Entry

- FR-5 The system shall allow a user to open an existing workflow in the visual editor by clicking Edit on its list row.
- FR-6 On entering the editor the system shall load the workflow configuration — its project statuses and its transitions — for the selected workflow.
- FR-7 The system shall allow a user to return from the editor to the workflow list.

3.3 Status (Node) Management

- FR-8 The system shall render each project status as a node on the editor canvas, labelled with the status name.
- FR-9 The system shall allow a user to create a new status by name; on success the status appears as a new node.
- FR-10 The canvas layout shall be responsive — the number of nodes per row adapts to the available width (4 / 3 / 2 columns for desktop / tablet / mobile).

3.4 Transition Creation

- FR-11 The system shall let a user define a transition by clicking a source status node (“From”) and then a different target status node (“To”); clicking the already-selected source again deselects it.
- FR-12 On selecting a second status the system shall open a Create Transition dialog showing From → To and asking for the transition name.
- FR-13 A newly created transition shall be recorded as pending and rendered as a dashed arrow until it is activated.

3.5 Transition Visualization

- FR-14 The system shall render active transitions as solid curved arrows and pending transitions as dashed curved arrows, each pointing from the source node to the target node.
- FR-15 The system shall show a legend distinguishing Active Transition from Pending Transition.
- FR-16 The system shall display each transition’s name as a label along its arrow.

3.6 Transition Detail and Lifecycle

- FR-17 The system shall let a user open a transition’s detail panel by clicking its arrow, showing the transition name, Id, From status, To status, record status, and created date.
- FR-18 The system shall let a user activate a pending transition (Activate is offered only while the transition is pending).

- FR-19 The system shall let a user delete a transition after a confirmation prompt; deletion is a soft delete and the arrow is removed from the canvas.
- FR-20 The system shall let a user close the detail panel.

3.7 Validation Rules

- FR-21 The system shall let a user add a field-validation rule to a transition by choosing a ticket field (API name) and a validation type.
- FR-22 The system shall let a user expand a transition to view its validation rules; the rules are loaded on expand.

3.8 Workflow Update / Activation

- FR-23 The system shall let a user update the workflow, which activates every pending transition of that workflow in a single action and returns to the workflow list.

4. Validation Rules

4.1 Client-side (LWC validators)

- VR-1 A status name is required and must be 80 characters or fewer (validateStatusName).
- VR-2 A transition requires both a From and a To status, and the two must be different (validateTransition).
- VR-3 A transition name is required (validateTransitionName).
- VR-4 A validation rule requires both a ticket field and a validation type (validateTicketField, validateValidationType).
- VR-5 A transition must be selected before its validation details can be loaded (validateTransitionId).
- VR-6 Creating a workflow requires a non-blank name and a selected project; creating a transition requires a workflow to be open.

4.2 Server-side (controller input guards)

- VR-7 Every @AuraEnabled method blank-checks its required parameters and returns APIResponse(false, '<field> is required') before doing any work (e.g. workflowId, projectId, name, fromStatusId, toStatusId, workflowTransitionId, fieldName, type).
- VR-8 Referenced records must exist via DomainCorrectness.require* (requireProjectExists, requireWorkflowExists, requireStatusExists, requireTransitionExists) before any insert/update.

5. Business Rules

5.1 Workflows & projects

- BR-1 A workflow belongs to exactly one project; the workflow list shows only that project's workflows (loadWorkflowsByProject).
- BR-2 A status belongs to the project and is shared across that project's workflows; the editor's node set is the project's statuses plus any status referenced by a transition.

5.2 Transition lifecycle

- BR-3 A transition is created in the pending record status and has no effect on ticket movement until it is active.
- BR-4 A transition becomes active either individually (Activate) or in bulk when the workflow is updated; Update Workflow activates all of that workflow's pending transitions at once.
- BR-5 Deleting a transition is a soft delete (record status set to deleted): the arrow disappears but the underlying record is retained.

5.3 Validation rules

- BR-6 A validation rule (ValidateField__c) belongs to a single transition and records the ticket field plus the validation type that a ticket must satisfy to take that transition.
- BR-7 The selectable fields and types mirror the ValidateField__c restricted picklists (FieldName__c / Type__c), so the page only offers values the backend accepts.

5.4 Data hygiene

- BR-8 Workflow, status, transition, and validation reads exclude soft-deleted rows, per the project's soql-exclude-deleted rule.

6. Use Case Descriptions

UC-1 — Open a workflow for editing

- Actor: Project Member
- Pre-conditions: User is authenticated; a projectId exists in localStorage.
- Main flow:

1. The page loads and lists the project's workflows (loadWorkflowsByProject). 2. User clicks Edit on a workflow. 3. The workflow Id is stored and getWorkflow loads its statuses and transitions. 4. The visual editor renders the nodes and arrows.

- Alternative flows:
- A1 (no project): No projectId → Choose-Project splash; on projectchosen the project is stored and the list loads.
- A2 (no workflows): The list is empty → empty-state message; the user creates one (UC-2).
- A3 (load error): Apex returns success=false or throws → error toast.

UC-2 — Create a workflow

- Actor: Project Member
- Pre-conditions: A project is selected; the workflow list is shown.
- Main flow:

1. User clicks Create New Workflow and enters a name. 2. createWorkflow(name, projectId) persists it. 3. On success the new workflow Id is stored and the editor opens on the new (empty) workflow.

- Alternative flows:
- A1 (blank name): Required-field validation; no call.

- A2 (server error): Error toast; the modal stays open.

UC-3 — Create a status

- Actor: Project Member
- Pre-conditions: The editor is open.
- Main flow:

1. User clicks Create Status and enters a name. 2. createStatus(name, projectId) persists it. 3. The new status is added to the canvas as a node.

- Alternative flows:
- A1 (invalid name): Required / length validation (≤80 chars); no call.

UC-4 — Create a transition

- Actor: Project Member
- Pre-conditions: The editor is open with at least two statuses.
- Main flow:

1. User clicks a source status node (From), then a different target node (To). 2. The Create Transition dialog opens showing From → To. 3. User enters a name and confirms. 4. addWorkflowTransition(workflowId, name, fromStatusId, toStatusId) persists it. 5. The transition appears as a pending (dashed) arrow.

- Alternative flows:
- A1 (same status / missing side): “From and To cannot be the same status” / “Both from and to statuses are required”; no call.
- A2 (blank name): “Transition name is required”; no call.

UC-5 — View, activate, or delete a transition

- Actor: Project Member
- Pre-conditions: A transition arrow is visible.
- Main flow (any one of):
- View → click the arrow → the detail panel shows name, Id, from/to, record status, created date.
- Activate (pending only) → activateWorkflowTransition → the arrow becomes active (solid).
- Delete → confirm → deleteWorkflowTransition (soft delete) → the arrow is removed.
- Alternative flows:
- A1 (cancel delete): The confirmation is dismissed; nothing changes.
- A2 (server error): Error toast; the panel keeps its last good value.

UC-6 — Add and view validation rules

- Actor: Project Member
- Pre-conditions: The transition detail panel is open.
- Main flow:

1. User clicks Add Validation Rule, chooses a Field API Name and a Validation Type, and confirms. 2. addValidateField(workflowTransitionId, fieldName, type) persists the rule and it is folded onto the transition. 3. User clicks Show Validation Details to expand and list the transition’s rules (loadValidateFields).

- Alternative flows:
- A1 (missing field/type): Validation toast; no call.
- A2 (no transition selected on expand): “A transition must be selected before loading its validation details.”

UC-7 — Update the workflow (activate pending)

- Actor: Project Member
- Pre-conditions: The editor is open and the workflow has one or more pending transitions.
- Main flow:

1. User clicks Update Workflow. 2. updateFullWorkflowTransition(workflowId) activates all pending transitions and reports the count. 3. The page returns to the workflow list.

- Alternative flows:
- A1 (no workflow id): “Workflow ID not found” error toast; no call.
- A2 (server error): Error toast; the editor stays open.

7. Non-Functional Notes

- Derived UI state: the LWC keeps one principal de-normalized state (workflowData = the workflow with its statuses and transitions) and derives every displayed value — the sorted status list, node positions, the SVG view box, the active/pending arrow geometry, the selected transition — via getters and pure utility functions, never parallel tracked fields.
- Responsive geometry: a ResizeObserver on the editor container recomputes the layout config when the width changes meaningfully (> 20px), so the graph re-flows between 4 / 3 / 2 columns without a server round-trip.
- Lazy validation reads: a transition’s validation rules are loaded only when the detail is expanded, behind a dedicated wire gate, so selecting a transition does not by itself fetch its rules.
- Type-safe option lists: the validation field/type comboboxes are sourced from constants that mirror the ValidateField__c restricted picklists, so the page emits exactly the API values the insert expects.
- Soft delete everywhere: transitions (and the other workflow records) are soft-deleted; reads filter them out.